

I Dizionari in Python

La struttura dati definitiva
per le coppie chiave-valore.

Prof. Antonio ADINOLFI

Un manuale visivo.



Chiavi (Keys)

- **Uniche** (Niente duplicati)
- **Immutabili** (Stringhe, Numeri, Tuple)



Valori (Values)

- **Qualunque tipo** (Interi, Liste, Dizionari)
- **Modificabili** (Mutable)



Ordinati per inserimento (dalla versione Python 3.7+)

Creating Python Dictionaries

1. Letterale { }

```
1 studente = {  
2     "nome": "Luca",  
3     "età": 17,  
4     "classe": "4A"}
```

2. Costruttore dict()

```
1 auto = dict(  
2     marca="Ford",  
3     modello="Mustang",  
4     anno=1964)
```

3. Dizionario Vuoto

```
1 d = {}
```




Accesso Diretto

```
print(studente["indirizzo"])
```

Causa un **KeyError** se la chiave non esiste.



Il Metodo .get()

```
print(studente.get("indirizzo",  
                    "Non disponibile"))
```

Più sicuro. Restituisce un valore **di default** se la chiave è assente.

Modificare i Dizionari Python



Aggiungere o Aggiornare

```
studente['età'] = 18  
studente['indirizzo'] = "Via Roma"
```



Rimuovere un Elemento

```
studente.pop("classe")  
del studente['età']
```



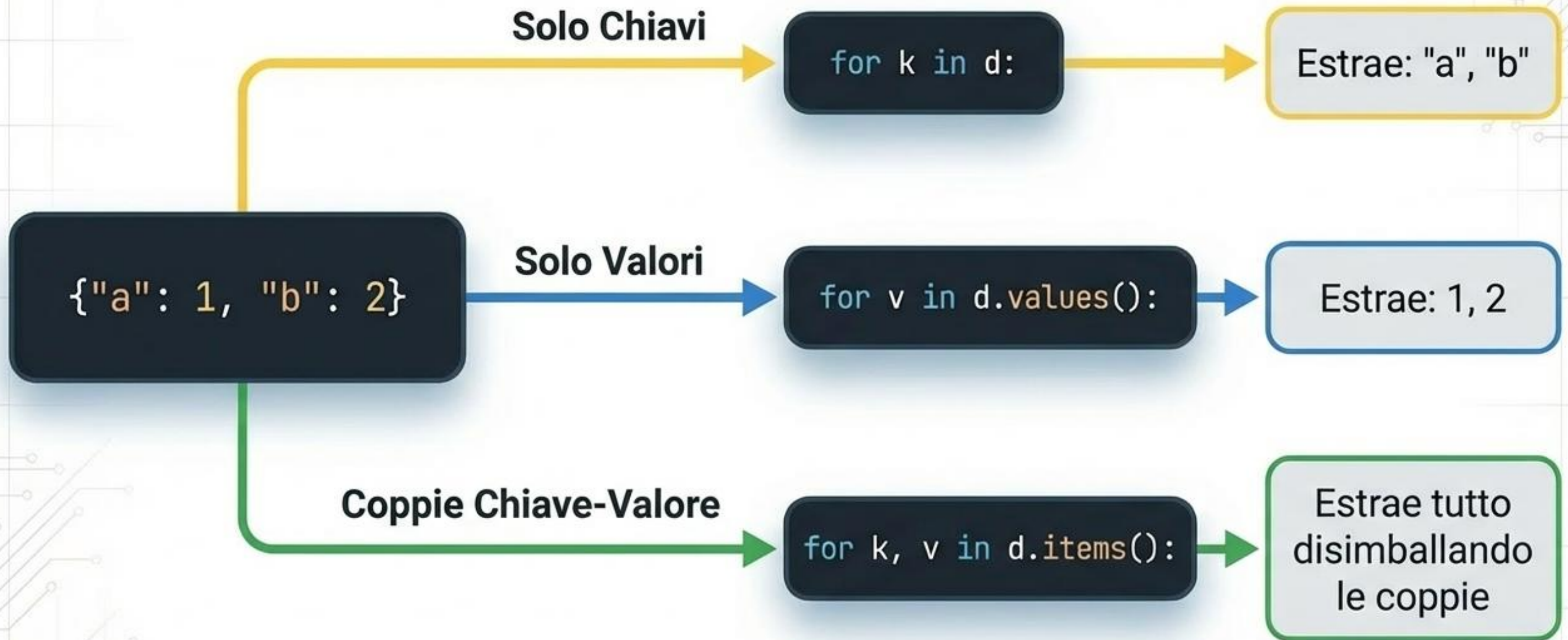
Svuotare Tutto

```
studente.clear()
```



Natura Dinamica

I dizionari sono strutture altamente modificabili (Mutable) in tempo reale.



Toolbox dei Metodi Essenziali



`.keys()`

Restituisce tutte le chiavi. (Es: `d.keys()`)



`.values()`

Restituisce tutti i valori. (Es: `d.values()`)



`.items()`

Restituisce coppie chiave-valore.
(Es: `d.items()`)



`.update()`

Aggiorna fondendo un altro dizionario.
(Es: `d.update({"c": 3})`)



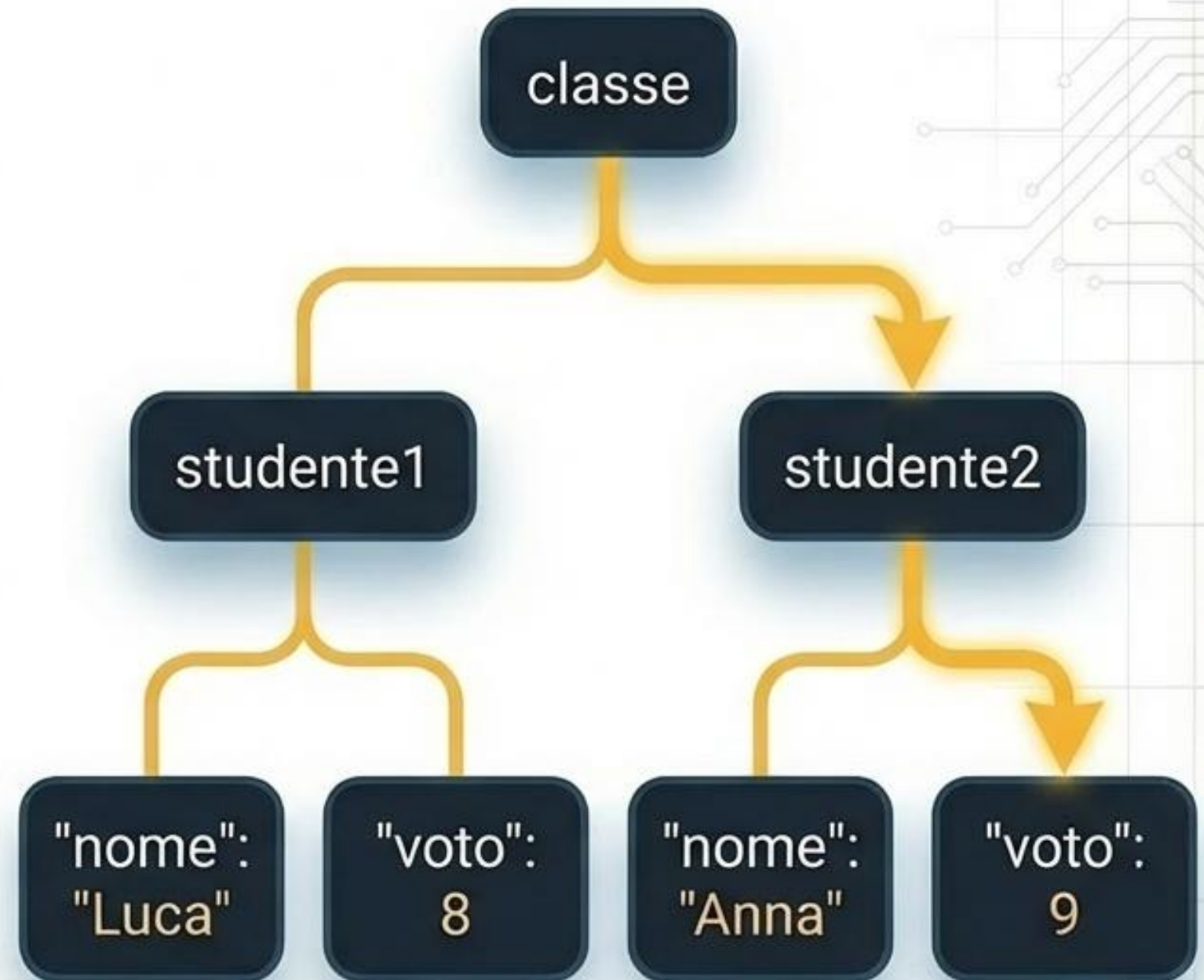
`.fromkeys()`

Crea un dizionario da una lista di chiavi.
(Es: `dict.fromkeys(["a", "b"], 0)`)

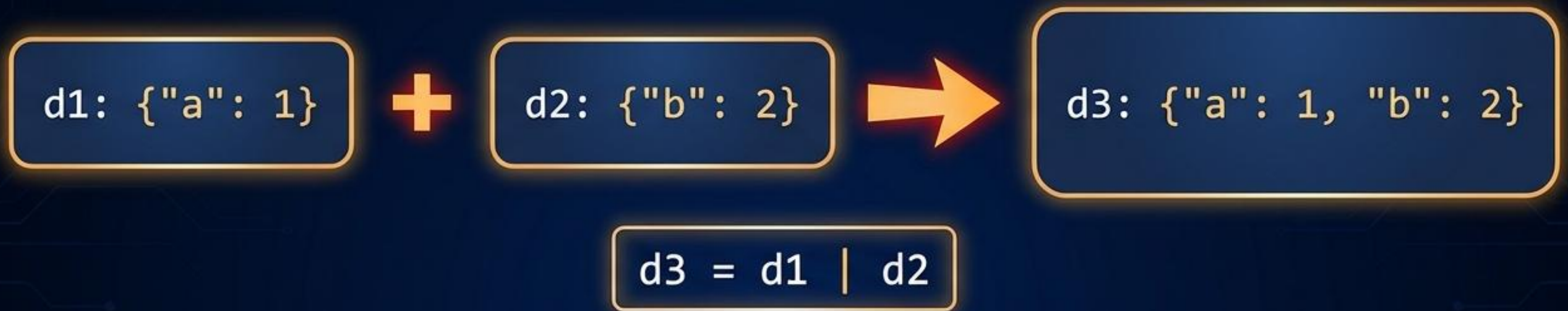
Strutture Annidate (Nested Dicts)

```
classe = {  
    "studente1":  
        {"nome": "Luca", "voto": 8}  
    },  
    "studente2":  
        {"nome": "Anna", "voto": 9}  
}
```

```
print(classe["studente2"]["voto"])
```



Operatore di Unione (|)



Aggiornamento In-place (|=)



Dictionary Comprehension: Potenza e Sintesi



```
quadrati = {x: x*x for x in range(1, 6)}
```


Ricetta 1: Registro Voti

```
voti = {"Luca": [8, 7, 9],  
        "Anna": [9, 9, 10]}
```

```
media_luca =  
sum(voti["Luca"]) /  
len(voti["Luca"])
```

Ricetta 2: Conteggio Frequenze

```
testo = "banana"; freq = {}
```

```
for c in testo:  
    freq[c] =  
    freq.get(c, 0) + 1
```

Pattern
fondamentale:
.get() crea la
chiave a 0 se non
esiste, poi
somma 1.

Che tipo di dati devi gestire?

Sequenziale o per
posizione numerica?

Usa una **Lista []**

Dati unici
valore

Usa un **Set {}**

senza un valore
associato?

Accesso rapido tramite
nome/attributo?

Usa un **Dizionario
{k:v}**

**I dizionari sono la scelta ideale per modellare oggetti
reali con attributi ben definiti.**

La Mappa del Tesoro (Cheat Sheet)

Creare

```
d = {}  
d = dict(a=1, b=2)  
{x: x*2 for x in range(5)}
```

Accedere

```
valore = d["chiave"]  
sicuro = d.get("chiave", default)
```

Modificare

```
d["nuova"] = 10  
d.update(altro_diz)  
d.pop("vecchia")  
d.clear()
```

Iterare

```
for k in d.keys():  
    for v in d.values():  
        for k, v in d.items():
```


3 Cose da Ricordare



Velocità.

Accesso istantaneo ai dati tramite chiavi univoche.



Flessibilità.

I valori possono contenere qualsiasi tipo di dato, creando gerarchie complesse.



Fondamentali.

Sono il cuore pulsante del linguaggio Python e della sua architettura.